Support Vector Machine (SVM) classifier using the Iris dataset

Python codes for Support Vector Machine (SVM) classifier using the Iris dataset and then testing it with an unknown sample (Jupyter Notebook)

1. Import Libraries

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
```

2. Load the Iris Dataset

In [2]:
```
iris = load_iris()

X = iris.data
y = iris.target

print("Feature names:", iris.feature_names)
print("Target names:", iris.target_names)
```

```
Feature names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal
width (cm)']
Target names: ['setosa' 'versicolor' 'virginica']
```

3. Split the Dataset

In [3]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

```
Training samples: 120
Testing samples: 30
```

4. Feature Scaling

In [4]:
```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

5. Train SVM Classifier

In [5]:
```
svm = SVC(kernel='rbf', probability=True, random_state=42)
svm.fit(X_train, y_train)
```

Out[5]:
```
SVC(probability=True, random_state=42)
```

6. Evaluate the Model

In [6]:
```
y_pred = svm.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n",
      classification_report(y_test, y_pred, target_names=iris.target_names))
```

```
Accuracy: 1.0

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

7. Predict Unknown Sample

In [7]:
```
# Unknown flower (sepal length, sepal width, petal length, petal width)
unknown_sample = np.array([[5.1, 3.5, 1.4, 0.2]])

# Must apply same scaling
unknown_sample_scaled = scaler.transform(unknown_sample)

prediction = svm.predict(unknown_sample_scaled)
predicted_class = iris.target_names[prediction[0]]

print("Predicted class for unknown sample:", predicted_class)
```

```
Predicted class for unknown sample: setosa
```

8. Test Multiple Unknown Samples

In [8]:
```
unknown_samples = np.array([
    [6.7, 3.1, 4.7, 1.5],
    [7.2, 3.6, 6.1, 2.5],
    [5.0, 3.4, 1.5, 0.2]
])

unknown_samples_scaled = scaler.transform(unknown_samples)

predictions = svm.predict(unknown_samples_scaled)

for i, pred in enumerate(predictions):
    print(f"Sample {i+1} → {iris.target_names[pred]}")
```

```
Sample 1 → versicolor
Sample 2 → virginica
Sample 3 → setosa
```